

Hardwired Control Unit Design

Lecture Notes — Week 6

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

6.1 Introduction

Week 5 built two hardware datapaths — a single-bus organization and a three-bus organization — and traced the same instruction, $R1 \leftarrow R2 + R3$, through each of them by hand. On the single-bus datapath the transfer took three control steps, T_1, T_2, T_3 , using two hidden registers Y and Z to buffer the bus; on the three-bus datapath the same transfer completed in a single control step, at the cost of more wiring. Both traces, however, shared a gap left open on purpose: T_1, T_2, T_3 were written by hand, on paper. Nothing in the hardware built in Week 5 decides “it is now T_2 ” or “assert Z_{in} this cycle.” Something inside the CPU must make that decision automatically, for every instruction, every cycle — and identifying and building that something is this chapter’s subject.

This chapter follows one thread through three questions: what the control unit reads and decides every clock cycle (Section 6.2); what hardware generates the control-step sequence $T_1, T_2, \dots$ automatically instead of it being written by hand (Section 6.4); and, given an instruction’s opcode and the current step, how the actual logic gates that assert the right control signals are derived (Sections 6.5–6.6). Throughout, the identical running example from Week 5, $R1 \leftarrow R2 + R3$ on the single-bus datapath, is reused, and the actual control unit that drives it is built, so that “the hardware decides” becomes gates and Boolean equations rather than a hand-wave.

6.2 Functions of the Control Unit

6.2.1 Motivation

In Week 5’s trace, register Y loaded from the bus at T_1 , and register Z loaded from the ALU at T_2 — never the other way around. Something decided that ordering. Not the ALU, which only computes what it is told to compute; not the bus, which only carries whatever is driven onto it; not the register file, which only loads or drives a register when a signal tells it to. Every one of those components is *passive* — each moves data only when explicitly told to, and none of them “knows” what step of the instruction is currently executing. Something else has to watch the instruction and the clock, and tell every other component what to do, every single cycle.

Definition (Control Unit). *The control unit is the hardware that, every clock cycle, examines the current instruction and the current control step, and **asserts the control signals** that make every other component — registers, ALU, memory, buses — do the right thing for that instruction at that step.*

It is important to be precise about what the control unit does *not* do: it does not compute results itself, since that is the ALU’s job; and it does not store data itself, since that is the job of

the registers and memory. The control unit’s entire responsibility is deciding *when* each of those components acts, and *which* of them acts at any given moment.

6.2.2 Inputs and Outputs

Before designing anything internal to the control unit, it helps to fix its interface — what goes in, and what comes out. Picture the control unit as a single black box, with its inputs entering from the left and its outputs leaving from the right, each output aligned on the same row as the input that most directly determines it. Four signals enter: the opcode currently sitting in the instruction register (IR), the condition-code flags (Z , N , C , V) reporting the outcome of the last ALU operation, the current control step T_i , and the clock/reset lines that drive the control unit’s own timing. Four categories of signal leave, each row-aligned with the input that drives it: register gating signals ($X_{\text{in}}/X_{\text{out}}$ for every register X), the ALU’s function-select lines, the memory read/write lines, and a next-step signal that tells the sequencing hardware to advance. This input/output contract follows the standard treatment of control-unit design (Hamacher, *Computer Organization*, Ch. 7) [1], and it is the contract every design choice in the rest of this chapter has to satisfy.

6.2.3 Notation: Register-Gating Control Signals

The control unit’s most frequent job is telling registers when to load and when to drive the bus, so this notation is introduced here: $X_{\text{in}} = 1$ means “gate register X to *load* whatever is currently on the bus, this cycle,” and $X_{\text{out}} = 1$ means “gate register X to *drive* its contents *onto* the bus, this cycle.” Applying this notation to Week 5’s single-bus trace of $R1 \leftarrow R2 + R3$ makes the connection concrete: at T_1 , $R2_{\text{out}}$ and Y_{in} fire together, so $R2$ drives the bus and Y latches whatever appears there; at T_2 , $R3_{\text{out}}$ fires (putting $R3$ on the bus), the ALU is set to add, and Z_{in} fires so the sum of Y and the bus value latches into Z ; at T_3 , Z_{out} and $R1_{\text{in}}$ fire together, so Z drives the bus and $R1$ latches the final result. These three lines are exactly the control signals the control unit must generate automatically.

6.3 Hardwired versus Microprogrammed Control

There are two fundamentally different ways to build the hardware that maps (opcode, step) pairs onto control signals. **Hardwired control** builds that mapping directly out of logic gates: fast, because the signals appear combinational with no extra delay, but the wiring is fixed once the chip is built. **Microprogrammed control** instead stores the control signals for each step as “microinstructions” inside a small memory, and steps through them with a microprogram counter: slower per step, because reading a stored word costs a memory access, but far easier to change, because updating a microprogram means rewriting memory contents rather than rewiring gates. This chapter builds the classical, gate-level hardwired approach in full (Sections 6.4–6.6); Week 7 rebuilds the identical control unit as microprogrammed control, so the two can be compared directly on the same worked example.

6.4 The Sequence Counter and Timing Generation

6.4.1 The Sequence Counter (SC)

The hardware that keeps track of “which step we are currently on” is a small counter register called the **sequence counter (SC)**, which holds the number of the current control step. A **step decoder**

converts SC 's binary value into one-hot signals $T_1, T_2, T_3, \dots$, so that exactly one T_i line is active in any given cycle. Structurally, this is a two-stage pipeline of hardware: SC is a 4-bit counter register, feeding its binary count into a 4×16 step decoder, which fans out into individual one-hot output lines — the decoder takes the counter's compact binary encoding and expands it into exactly one active line per step, which is precisely the encoding every other part of the control unit needs, since a signal like $Y_{in} = D_{ADD} \cdot T_1$ is far easier to wire from a one-hot T_1 line than from SC 's raw binary count (Figure 6.1).

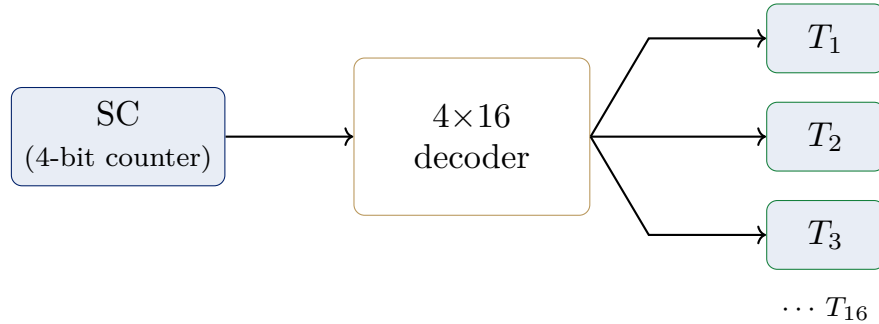


Figure 6.1: The sequence counter feeding a step decoder — cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-6, p.137. This course numbers steps from T_1 (Mano's own text numbers from T_0), so a 4-bit/16-state counter yields $T_1, \dots, T_{16}$ here rather than Mano's $T_0, \dots, T_{15}$ [2].

This SC-plus-decoder pattern is the classical sequence-counter method for control-step generation.

6.4.2 How SC Advances

By default, every clock pulse simply increments the counter: $SC \leftarrow [SC] + 1$, which moves the step decoder's one-hot output to the next T_i automatically, with no external intervention needed. But at the *last* step of an instruction, the control unit instead asserts a special signal, SC_{clear} , which resets $SC \leftarrow 0$ and returns the decoder to T_1 — signaling that the current instruction is finished and it is time to begin fetching the next one. This reset is not optional bookkeeping: without SC_{clear} , SC would simply keep incrementing — T_4, T_5 , and onward, forever. The step decoder would dutifully produce one-hot lines for these later steps, but since no instruction's state table defines any signals for T_4 or beyond, the datapath would simply idle: nothing would fire, no new instruction would ever be fetched, and the CPU would freeze after the first instruction completed its useful work. SC_{clear} is therefore precisely what allows the control unit to cycle through a program's instructions at all, rather than freezing after the very first one.

Example 6.4.1 (Worked Trace Revisited: Where SC Points). *Laying SC 's actual numeric value alongside the RTL trace for $R1 \leftarrow R2+R3$ makes the mechanism concrete.*

Step	SC value	Active line	RTL
T_1	0	$T_1 = 1$	$Y \leftarrow [R2]$
T_2	1	$T_2 = 1$	$Z \leftarrow [Y] + [R3]$
T_3	2	$T_3 = 1, SC_{clear} = 1$	$R1 \leftarrow [Z], \text{ then } SC \leftarrow 0$

SC together with the step decoder is the hardware that turns “now it's T_2 ” from something written by hand into something the circuit produces on its own, every cycle, with no human intervention. For this instruction, SC_{clear} fires at T_3 , the last step.

6.5 Opcode Decoding and the Control Matrix

6.5.1 The Missing Piece: Opcode Decoding

SC answers *when* — which T_i is active. But it says nothing about *which* instruction is running: Y_{in} should fire at T_1 for an ADD, while a LOAD instruction needs something entirely different at its own T_1 . SC alone has no way to distinguish these cases. The missing piece is a new notational device: D_i , a one-hot **opcode decoder** output, where $D_{ADD} = 1$ exactly when the IR holds the ADD opcode, and every other $D_j = 0$ at that moment. With this notation in hand, the key structural fact of hardwired control emerges: every control signal will turn out to depend on *both* an opcode line (D_i) and a step line (T_j) together — never on either alone.

6.5.2 Two Decoders Feed the Control Matrix

Putting the two decoders side by side clarifies the overall shape of the design (Figure 6.2). The instruction register IR feeds a 3×8 operation decoder, which produces the one-hot $D_0, \dots, D_7$ lines identifying which instruction is currently executing, and also loads the I flip-flop from IR bit 15 (the indirect-addressing mode bit). In parallel, the sequence counter SC feeds a 4×16 step decoder, producing the one-hot $T_1, \dots, T_{16}$ lines identifying which control step is currently active. All three signals — the D_i lines, the I flip-flop, and the T_j lines — converge on a single combinational block called the **control matrix**, which produces the actual control signals as output: register-gating signals such as Y_{in} and Z_{in} on one output path, and SC_{clear} on another.

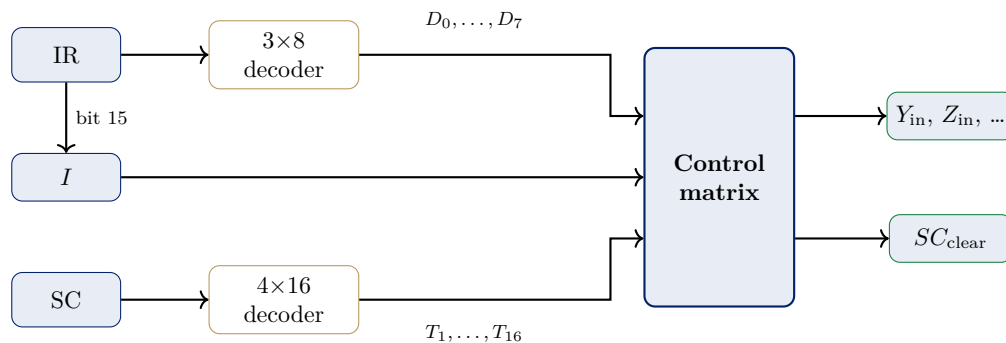


Figure 6.2: Two decoders feeding the control matrix — cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-6, p.137. IR bits 0–11 also feed the control logic gates directly (omitted above for clarity) [2].

The key claim this figure makes precise is that every control signal is a Boolean function of the D_i lines, the I flip-flop, and the T_j lines together — the control matrix is nothing more than the circuitry that implements that Boolean function for every signal the datapath needs.

6.6 Deriving the Control Equations

6.6.1 The State Table

The bridge from the RTL trace already known to the Boolean equations derived next is a state table.

Definition (State Table). A state table lists, for each control step, which control signals are active. A signal is asserted only when both the relevant opcode line $D_i = 1$ and the listed step $T_j = 1$ hold simultaneously.

For $R1 \leftarrow R2+R3$, the state table has three rows:

Step	RTL	Control signals active
T_1	$Y \leftarrow [R2]$	$R2_{out}, Y_{in}$
T_2	$Z \leftarrow [Y] + [R3]$	$R3_{out}, ALU=add, Z_{in}$
T_3	$R1 \leftarrow [Z]$	$Z_{out}, R1_{in}, SC_{clear}$

6.6.2 From State Table to Boolean Equations

Reading straight off the state table for **ADD** (using the opcode line D_{ADD}) produces one equation per control signal:

$$Y_{in} = D_{ADD} \cdot T_1, \quad Z_{in} = D_{ADD} \cdot T_2, \quad R1_{in} = D_{ADD} \cdot T_3, \quad SC_{clear} = D_{ADD} \cdot T_3.$$

Each equation reads exactly the way it looks: “this signal fires when this instruction is running *and* we are at this step.” This is only half the story for any signal shared across multiple instructions — if a *different* instruction also needs, say, Z_{in} at one of its own steps, that instruction’s term gets **OR**’d into the very same equation.

Example 6.6.1 (Deriving Z_{in} for Two Instructions). Suppose **SUB** also uses register Z to hold its result, at its own T_2 — a reasonable assumption, since **SUB** has an identical three-step shape to **ADD**, differing only in which ALU function is selected at T_2 .

1. From **ADD**’s state table, Z_{in} fires at $D_{ADD} \cdot T_2$.
2. From **SUB**’s state table, Z_{in} also fires at $D_{SUB} \cdot T_2$.
3. Any instruction whose state table lists Z_{in} at T_2 contributes a term, so the two terms combine by **OR**:

$$Z_{in} = (D_{ADD} \cdot T_2) + (D_{SUB} \cdot T_2) = T_2 \cdot (D_{ADD} + D_{SUB}).$$

This exposes the general pattern behind every control signal in a hardwired design: each signal’s equation is an **OR**, taken across every instruction that uses it, of (opcode line $\cdot$ step line) terms.

6.6.3 The Control Matrix as Actual Logic Gates

The Boolean equation for Z_{in} derived above is not just algebra — it is literally a circuit diagram waiting to be drawn. Two AND gates realize the two product terms: the first takes D_{ADD} and T_2 as inputs and produces $D_{ADD} \cdot T_2$; the second takes D_{SUB} and T_2 and produces $D_{SUB} \cdot T_2$. Both AND-gate outputs feed a single OR gate, whose output is the final Z_{in} signal. Two AND gates, one per instruction that needs Z_{in} at T_2 , feeding one OR gate that combines them — this AND-OR structure is literally what the word “hardwired” means in hardwired control: the Boolean equation and the physical gate layout are the same object, viewed two ways.

6.6.4 The Full Hardwired Control Unit

Assembling every piece built so far into one capstone diagram shows the complete hardwired control unit (Figure 6.3). On the input side, four sources feed the control matrix: the instruction register IR feeds a 3×8 decoder, which produces the D_i lines; the condition-code flags feed the matrix directly; the clock feeds the matrix directly; and the sequence counter SC feeds a 4×16 decoder, which produces the T_j lines. On the output side, the control matrix drives two categories of signal: register gating, ALU function select, and memory read/write signals, which together drive the datapath built in Week 5; and an $SC_{\text{clear/advance}}$ signal, which feeds back to control whether SC advances normally or resets to zero. This feedback loop is what closes the system: the control matrix's own output determines whether the sequence counter that feeds the control matrix continues counting or restarts, cycle after cycle, for every instruction the CPU executes.

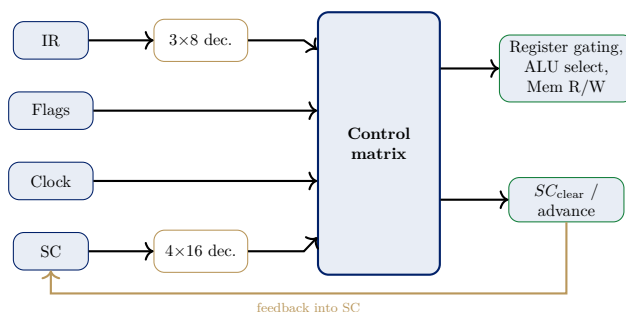


Figure 6.3: The complete hardwired control unit — cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-6, p.137 (core: two decoders + SC + control logic gates), extended with explicit Flags/Clock inputs and the SC feedback loop [2, 1].

Example 6.6.2 (Adding a New Instruction). Suppose a new *MUL* instruction is added to the instruction set, and it too uses register Z at its own T_2 . What has to change in the hardwired control unit just built? Three things, all physical:

1. The opcode decoder needs an entirely new output line, D_{MUL} , since the existing decoder has no way to signal “this is a *MUL* instruction.”
2. Every equation for a signal that *MUL* needs — Z_{in} among them — must be **re-derived** from scratch to include the new term.
3. A new AND gate, computing $D_{MUL} \cdot T_2$, must be physically wired into the existing OR gate that already combines *ADD*’s and *SUB*’s contributions to Z_{in} .

None of this is a software change: it is new gates, added to the actual chip, for every single signal the new instruction touches.

6.7 Trade-offs, Scaling, and Summary

6.7.1 Trade-offs of Hardwired Control

Hardwired control’s advantages and costs both follow directly from the fact that it is pure combinational logic. On the positive side, control signals appear the *same cycle* they are needed, with no extra memory-access delay, and there is no wasted circuitry: the gates that exist are exactly

the ones the instruction set architecture needs, nothing more. On the negative side, adding, removing, or fixing a bug in even one instruction means re-deriving Boolean equations and *physically rewiring* logic gates, exactly as the previous example demonstrated; and the control matrix as a whole grows with the number of instructions *times* the number of signals, so verifying it by hand becomes rapidly harder as the instruction set grows.

6.7.2 Scaling Limits

The worked example running through this chapter was deliberately small: one instruction (ADD), three steps, and a handful of signals, small enough that its equations fit on a single page. A real instruction set architecture, by contrast, has dozens to hundreds of instructions, each contributing its own state table, and each contributing AND terms to *every* shared signal's OR equation. Every new instruction added to the ISA, and every microarchitectural bug fix, means re-deriving Boolean equations and re-wiring gates by hand — so design and verification cost grows sharply as the instruction set grows in complexity [3].

6.7.3 Toward Microprogrammed Control

This scaling problem motivates a genuinely different idea: what if, instead of wiring Y_{in} , Z_{in} , $R1_{in}$, and every other signal directly into logic gates, these were simply *written down* — one row per control step, stored in a small memory — and a counter stepped through the rows? Under this scheme, adding a new instruction becomes a matter of writing new rows into memory, not redesigning gates from scratch. The trade-off flips accordingly: each step becomes slightly slower, since reading a stored control word costs a memory access, but the design becomes far easier to build, verify, and modify. This is **microprogrammed control**, taken up in Week 7, which rebuilds this same $R1 \leftarrow R2+R3$ control unit using that approach.

6.7.4 Effect of Bus Organization on Control Complexity

Control-step count is not an independent design parameter — it depends directly on the bus organization chosen in Week 5. On the single-bus datapath, three states (T_1 , T_2 , T_3) are needed, which means SC needs 2 bits, the step decoder has three outputs, and ADD's state table has three rows. On the three-bus datapath, only one state (T_1) is needed, which means SC barely needs to exist at all, and ADD's state table collapses to a single row. Fewer control steps mean a smaller state table and fewer AND/OR gates in the control matrix — the hardware cost simply moves from control logic into bus wiring instead. Neither the datapath choice made in Week 5 nor the control unit choice made in this chapter is free; every design decision moves cost somewhere else on the chip, never eliminating it.

6.7.5 Summary

The **control unit** is the hardware that reads (opcode, step, flags) every cycle and asserts control signals for every other component to act on. The **sequence counter (SC)**, together with a step decoder, generates $T_1, T_2, \dots$ automatically; SC_{clear} resets the counter back to T_1 at an instruction's last step. The **opcode decoder** produces one-hot D_i lines, and every control signal is an OR, taken across instructions, of $(D_i \cdot T_j)$ terms — the **state table** is what makes this systematic and mechanical rather than ad hoc. **Hardwired control** follows the pipeline state table $\rightarrow$ Boolean equations $\rightarrow$ AND/OR gates: fast, but rigid, since new instructions mean new gates, wired by

hand. This rigidity is exactly what motivates **microprogrammed control**, taken up in Chapter 7, which stores control words in memory instead of wiring them into gates.

References

- [1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.
- [2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.
- [3] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.